

HW #4 — Autoencoder for MNIST Dimensionality Reduction

Course: Deep Learning for Computer Vision (EN525.733.8VL)

Due Date: March 5, 2026

Assignment Overview

In this assignment we build a **3-layer MLP autoencoder** to perform dimensionality reduction (compression) on the MNIST handwritten digit dataset. The autoencoder learns to compress each 784-pixel image into a smaller latent representation and then reconstruct the original image from that compressed code.

Tasks

- 1. **Train autoencoders** with varying bottleneck sizes (5, 10, 20, 30, 60, 120 hidden units). Evaluate reconstruction quality using Mean Squared Error (MSE). Plot learning curves (MSE vs. epoch) for training, validation, and test sets.
- 2. **Visualize encoder weights** for 10 and 20 hidden units — reshape each neuron's incoming weight vector into a 28×28 image to see what visual pattern each neuron has learned.

Approach & Hyperparameters Summary

Hyperparameter	Value	Rationale
Architecture	784 → N → 784 (3-layer MLP)	Symmetric encoder-decoder as specified in assignment
Hidden units (N)	5, 10, 20, 30, 60, 120	Range from extreme compression to mild compression
Activation function	Sigmoid (both layers)	Output in (0,1) matches normalized pixel range; hidden layer enforces compact nonlinear representation
Loss function	Mean Squared Error (MSE)	Standard pixel-level reconstruction loss
Optimizer	Adam (from <code>torch.optim</code>)	Adaptive learning rate per parameter; converges reliably without manual LR tuning
Learning rate	0.001 (1e-3)	Default Adam LR; works well for this architecture

Hyperparameter	Value	Rationale
Batch size	256	Good balance between training speed and gradient stability
Epochs	50	Sufficient for convergence on this task
Train/Val split	80% / 20% of 60,000 training images	As specified in assignment (48,000 train, 12,000 validation)
Framework	PyTorch	Deep learning library used for model definition, training, and GPU acceleration

1. Imports and Setup

We use **PyTorch** as our deep learning framework. Key packages:

- `torch.nn` — defines the neural network layers (Linear, Sigmoid) and loss function (MSELoss)
- `torch.optim` — provides the Adam optimizer
- `torch.utils.data` — DataLoader for efficient mini-batch training
- `matplotlib` — for generating all plots
- `struct` — for reading the raw MNIST binary files

The code automatically uses GPU (CUDA) if available, otherwise falls back to CPU.

```
In [1]: import os
import struct
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

%matplotlib inline

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cuda

2. Load MNIST Data

The MNIST dataset is stored in a custom IDX binary format. Each image file has a 16-byte header containing the magic number, number of images, number of rows (28), and number of columns (28), all as big-endian 32-bit unsigned integers. The pixel data follows as unsigned bytes.

We load the raw bytes, reshape to (N, 784) (flattening each 28×28 image), and **normalize pixel values from [0, 255] to [0, 1]** by dividing by 255. This normalization is important because:

- It matches the Sigmoid output range of our decoder (which outputs values in (0, 1))
- It helps the optimizer converge faster by keeping values in a well-scaled range

```
In [3]: def load_mnist_images(filepath):
        """Read an MNIST IDX3-ubyte image file and return flattened, normalized images.
        with open(filepath, 'rb') as f:
            magic, num_images, rows, cols = struct.unpack('>IIII', f.read(16))
            raw = np.frombuffer(f.read(), dtype=np.uint8)
            images = raw.reshape(num_images, rows * cols).astype(np.float32) / 255.0
        return images

def load_mnist_labels(filepath):
    """Read an MNIST IDX1-ubyte label file and return integer labels."""
    with open(filepath, 'rb') as f:
        magic, num_labels = struct.unpack('>II', f.read(8))
        raw = np.frombuffer(f.read(), dtype=np.uint8)
    return raw.astype(np.int64)

# Path to the MNIST data files
DATA_DIR = "archive"

train_images = load_mnist_images(os.path.join(DATA_DIR, "train-images.idx3-ubyte"))
train_labels = load_mnist_labels(os.path.join(DATA_DIR, "train-labels.idx1-ubyte"))
test_images = load_mnist_images(os.path.join(DATA_DIR, "t10k-images.idx3-ubyte"))
test_labels = load_mnist_labels(os.path.join(DATA_DIR, "t10k-labels.idx1-ubyte"))

print(f"Training set: {train_images.shape[0]} images, each {train_images.shape[1]}")
print(f"Test set: {test_images.shape[0]} images")
print(f"Pixel value range: [{train_images.min():.1f}, {train_images.max():.1f}])")
```

Training set: 60000 images, each 784 pixels (28x28 flattened)

Test set: 10000 images

Pixel value range: [0.0, 1.0]

3. Train / Validation Split

As specified in the assignment, we split the 60,000 training images into:

- **48,000 training samples** (80%)
- **12,000 validation samples** (20%)

We shuffle the data before splitting (with a fixed random seed for reproducibility) to ensure all digit classes are represented proportionally in both sets. The **10,000 test images** are kept entirely separate.

The validation set is used to monitor generalization during training — if validation MSE starts increasing while training MSE keeps decreasing, it would indicate overfitting.

```
In [4]: VAL_RATIO = 0.20
N_total = len(train_images)          # 60,000
N_val   = int(N_total * VAL_RATIO)   # 12,000
N_train = N_total - N_val            # 48,000

# Reproducible shuffle
np.random.seed(42)
shuffle_idx = np.random.permutation(N_total)
train_images = train_images[shuffle_idx]
train_labels = train_labels[shuffle_idx]

# Split: first N_val -> validation, rest -> training
val_images = train_images[:N_val]
val_labels = train_labels[:N_val]
train_images = train_images[N_val:]
train_labels = train_labels[N_val:]

print(f"Training set:  {train_images.shape[0]:,} images")
print(f"Validation set: {val_images.shape[0]:,} images")
print(f"Test set:      {test_images.shape[0]:,} images")
```

Training set: 48,000 images

Validation set: 12,000 images

Test set: 10,000 images

4. Create PyTorch DataLoaders

We wrap our numpy arrays in PyTorch `TensorDataset` and `DataLoader` objects for efficient mini-batch training.

Key detail: For an autoencoder, the target is the input itself — we want to reconstruct x from x . So each dataset yields (x, x) pairs. No separate labels are needed during training.

Batch size = 256 — this provides a good balance between:

- Training speed (larger batches = fewer gradient updates per epoch)
- Gradient quality (not too noisy, not too smooth)

```
In [5]: BATCH_SIZE = 256

def make_loader(images, shuffle=True):
    """Create a DataLoader that yields (x, x) pairs for autoencoder training."""
    tensor = torch.from_numpy(images)
    dataset = TensorDataset(tensor, tensor)
    return DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=shuffle,
                      num_workers=0, pin_memory=(device.type == "cuda"))

train_loader = make_loader(train_images, shuffle=True)
```

```

val_loader = make_loader(val_images, shuffle=False)
test_loader = make_loader(test_images, shuffle=False)

print(f"Batch size: {BATCH_SIZE}")
print(f"Training batches per epoch: {len(train_loader)}")

```

Batch size: 256

Training batches per epoch: 188

5. Autoencoder Architecture

The autoencoder is a **3-layer MLP** (multilayer perceptron) with the following architecture:

Input Layer (L1): 784 units ← flattened 28×28 image
 Hidden Layer (L2): N units ← bottleneck / latent representation
 Output Layer (L3): 784 units → reconstructed image

Encoder: Linear(784 → N) + Sigmoid

- The linear layer computes $y = W_1 \cdot x + b_1$ where W_1 has shape (N, 784)
- Sigmoid activation squashes each value to (0, 1), creating a compact nonlinear representation
- Row i of W_1 contains the 784 weights converging to hidden neuron i — reshaping this row to 28×28 reveals what visual pattern neuron i has learned to detect

Decoder: Linear(N → 784) + Sigmoid

- The linear layer computes $\hat{y} = W_2 \cdot a + b_2$ where W_2 has shape (784, N)
- Sigmoid output activation ensures reconstructed pixel values are in (0, 1), matching our normalized input range

Why Sigmoid?

- **Hidden layer:** Forces the latent code into a bounded range, enforcing compression
- **Output layer:** Natural choice since pixel values are in [0, 1] — prevents predictions outside valid range

```

In [6]: class Autoencoder(nn.Module):
        def __init__(self, hidden_units: int):
            super(Autoencoder, self).__init__()

            # Encoder: 784 -> hidden_units with Sigmoid activation
            self.encoder = nn.Sequential(
                nn.Linear(784, hidden_units),
                nn.Sigmoid()
            )

            # Decoder: hidden_units -> 784 with Sigmoid activation

```

```

        self.decoder = nn.Sequential(
            nn.Linear(hidden_units, 784),
            nn.Sigmoid()
        )

    def forward(self, x):
        """Full autoencoder pass: x -> encode -> decode -> x_hat"""
        latent = self.encoder(x)
        recon = self.decoder(latent)
        return recon

    def get_encoder_weights(self):
        """Return encoder weight matrix (hidden_units x 784) as numpy array."""
        return self.encoder[0].weight.data.cpu().numpy()

# Quick sanity check with 10 hidden units
test_model = Autoencoder(10).to(device)
print(f"Model architecture (10 hidden units):")
print(test_model)
total_params = sum(p.numel() for p in test_model.parameters())
print(f"\nTotal parameters: {total_params:,}")
del test_model

```

Model architecture (10 hidden units):

```

Autoencoder(
  (encoder): Sequential(
    (0): Linear(in_features=784, out_features=10, bias=True)
    (1): Sigmoid()
  )
  (decoder): Sequential(
    (0): Linear(in_features=10, out_features=784, bias=True)
    (1): Sigmoid()
  )
)

```

Total parameters: 16,474

6. Training & Evaluation Functions

Training Loop

For each epoch:

1. **Forward pass:** Feed a batch of images through the autoencoder to get reconstructions
2. **Compute loss:** Calculate MSE between original and reconstructed images
3. **Backward pass:** Compute gradients via backpropagation (`loss.backward()`)
4. **Update weights:** Adam optimizer adjusts weights in the direction that reduces loss (`optimizer.step()`)
5. **Evaluate:** Compute MSE on validation and test sets (without gradient computation for efficiency)

Loss Function: Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

This measures the average squared difference between each input pixel x_i and its reconstruction $\hat{x}_i$. Lower MSE = better reconstruction.

Optimizer: Adam

Adam (Adaptive Moment Estimation) from `torch.optim.Adam` maintains per-parameter adaptive learning rates using first and second moment estimates of the gradients. This generally converges faster and more reliably than vanilla SGD for this type of problem, without requiring manual learning rate scheduling. We use the default learning rate of **0.001**.

```
In [7]: def compute_mse_on_loader(model, criterion, loader):
    """Compute average MSE over all batches in a DataLoader (no gradient computation)"""
    total_loss = 0.0
    total_count = 0
    model.eval()
    with torch.no_grad():
        for x_batch, _ in loader:
            x_batch = x_batch.to(device)
            recon = model(x_batch)
            loss = criterion(recon, x_batch)
            total_loss += loss.item() * x_batch.size(0)
            total_count += x_batch.size(0)
    return total_loss / total_count

def compute_mse_for_digit(model, criterion, images, labels, digit=3):
    """Compute MSE only for samples of a specific digit class."""
    mask = (labels == digit)
    digit_imgs = torch.from_numpy(images[mask]).to(device)
    model.eval()
    with torch.no_grad():
        recon = model(digit_imgs)
        loss = criterion(recon, digit_imgs)
    return loss.item()

def train_autoencoder(hidden_units, num_epochs=50, lr=1e-3):
    """
    Train an autoencoder and return the model + per-epoch loss histories.

    Uses:
    - Loss: MSELoss (mean squared error)
    - Optimizer: Adam with learning rate = lr
    """
    model = Autoencoder(hidden_units).to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.MSELoss()
```

```

train_losses, val_losses, test_losses = [], [], []

print(f"\n{'='*60}")
print(f"Training autoencoder with {hidden_units} hidden units")
print(f"Optimizer: Adam | LR: {lr} | Epochs: {num_epochs} | Batch size: {BATCH_")
print(f"{'='*60}")

for epoch in range(1, num_epochs + 1):
    # --- Training phase ---
    model.train()
    running_loss = 0.0
    for x_batch, _ in train_loader:
        x_batch = x_batch.to(device)
        optimizer.zero_grad()
        recon = model(x_batch)
        loss = criterion(recon, x_batch)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * x_batch.size(0)

    train_loss = running_loss / len(train_loader.dataset)

    # --- Evaluation phase ---
    val_loss = compute_mse_on_loader(model, criterion, val_loader)
    test_loss = compute_mse_on_loader(model, criterion, test_loader)

    train_losses.append(train_loss)
    val_losses.append(val_loss)
    test_losses.append(test_loss)

    if epoch % 10 == 0 or epoch == 1:
        print(f" Epoch {epoch:3d}/{num_epochs} | "
              f"Train MSE: {train_loss:.5f} | "
              f"Val MSE: {val_loss:.5f} | "
              f"Test MSE: {test_loss:.5f}")

return model, train_losses, val_losses, test_losses

```

7. Train All Models (Q1 — Part 1)

We train **6 separate autoencoders**, one for each bottleneck size: **5, 10, 20, 30, 60, and 120 hidden units**.

Each model is trained for **50 epochs** with the same hyperparameters (Adam optimizer, LR=0.001, batch size=256). After training each model, we also compute the reconstruction MSE specifically for **digit "3"** from the test set, as required by the assignment.

Expected behavior:

- Fewer hidden units = more aggressive compression = higher reconstruction error
- More hidden units = the model can preserve more information = lower error

- At some point, adding more hidden units gives diminishing returns

```
In [8]: HIDDEN_UNITS_LIST = [5, 10, 20, 30, 60, 120]
        NUM_EPOCHS       = 50

        criterion = nn.MSELoss()

        all_results = {} # {h: (model, train_losses, val_losses, test_losses)}
        digit3_mse  = {} # {h: MSE on test images of digit "3"}

        for h in HIDDEN_UNITS_LIST:
            model, train_l, val_l, test_l = train_autoencoder(
                hidden_units=h,
                num_epochs=NUM_EPOCHS,
                lr=1e-3
            )
            all_results[h] = (model, train_l, val_l, test_l)

            mse3 = compute_mse_for_digit(model, criterion, test_images, test_labels, digit=
            digit3_mse[h] = mse3
            print(f" → Digit-3 Test MSE ({h} hidden units): {mse3:.6f}")
```

```
=====
Training autoencoder with 5 hidden units
Optimizer: Adam | LR: 0.001 | Epochs: 50 | Batch size: 256
=====
Epoch 1/50 | Train MSE: 0.17846 | Val MSE: 0.11569 | Test MSE: 0.11612
Epoch 10/50 | Train MSE: 0.06806 | Val MSE: 0.06769 | Test MSE: 0.06814
Epoch 20/50 | Train MSE: 0.06745 | Val MSE: 0.06716 | Test MSE: 0.06760
Epoch 30/50 | Train MSE: 0.06736 | Val MSE: 0.06706 | Test MSE: 0.06752
Epoch 40/50 | Train MSE: 0.06733 | Val MSE: 0.06704 | Test MSE: 0.06749
Epoch 50/50 | Train MSE: 0.06732 | Val MSE: 0.06704 | Test MSE: 0.06749
→ Digit-3 Test MSE (5 hidden units): 0.069595
```

```
=====
Training autoencoder with 10 hidden units
Optimizer: Adam | LR: 0.001 | Epochs: 50 | Batch size: 256
=====
Epoch 1/50 | Train MSE: 0.13916 | Val MSE: 0.08398 | Test MSE: 0.08447
Epoch 10/50 | Train MSE: 0.06757 | Val MSE: 0.06726 | Test MSE: 0.06770
Epoch 20/50 | Train MSE: 0.06737 | Val MSE: 0.06708 | Test MSE: 0.06752
Epoch 30/50 | Train MSE: 0.06734 | Val MSE: 0.06704 | Test MSE: 0.06749
Epoch 40/50 | Train MSE: 0.06732 | Val MSE: 0.06703 | Test MSE: 0.06749
Epoch 50/50 | Train MSE: 0.06396 | Val MSE: 0.06348 | Test MSE: 0.06379
→ Digit-3 Test MSE (10 hidden units): 0.067588
```

```
=====
Training autoencoder with 20 hidden units
Optimizer: Adam | LR: 0.001 | Epochs: 50 | Batch size: 256
=====
Epoch 1/50 | Train MSE: 0.10874 | Val MSE: 0.07166 | Test MSE: 0.07215
Epoch 10/50 | Train MSE: 0.06512 | Val MSE: 0.06449 | Test MSE: 0.06486
Epoch 20/50 | Train MSE: 0.05648 | Val MSE: 0.05595 | Test MSE: 0.05610
Epoch 30/50 | Train MSE: 0.05304 | Val MSE: 0.05274 | Test MSE: 0.05279
Epoch 40/50 | Train MSE: 0.04636 | Val MSE: 0.04604 | Test MSE: 0.04598
Epoch 50/50 | Train MSE: 0.04210 | Val MSE: 0.04163 | Test MSE: 0.04152
→ Digit-3 Test MSE (20 hidden units): 0.045197
```

```
=====
Training autoencoder with 30 hidden units
Optimizer: Adam | LR: 0.001 | Epochs: 50 | Batch size: 256
=====
Epoch 1/50 | Train MSE: 0.09763 | Val MSE: 0.06918 | Test MSE: 0.06961
Epoch 10/50 | Train MSE: 0.05944 | Val MSE: 0.05771 | Test MSE: 0.05783
Epoch 20/50 | Train MSE: 0.04160 | Val MSE: 0.04105 | Test MSE: 0.04095
Epoch 30/50 | Train MSE: 0.03468 | Val MSE: 0.03436 | Test MSE: 0.03416
Epoch 40/50 | Train MSE: 0.03167 | Val MSE: 0.03158 | Test MSE: 0.03133
Epoch 50/50 | Train MSE: 0.02930 | Val MSE: 0.02922 | Test MSE: 0.02892
→ Digit-3 Test MSE (30 hidden units): 0.030969
```

```
=====
Training autoencoder with 60 hidden units
Optimizer: Adam | LR: 0.001 | Epochs: 50 | Batch size: 256
=====
Epoch 1/50 | Train MSE: 0.08478 | Val MSE: 0.06633 | Test MSE: 0.06672
Epoch 10/50 | Train MSE: 0.03124 | Val MSE: 0.03021 | Test MSE: 0.02994
Epoch 20/50 | Train MSE: 0.01982 | Val MSE: 0.01963 | Test MSE: 0.01922
Epoch 30/50 | Train MSE: 0.01621 | Val MSE: 0.01620 | Test MSE: 0.01574
```

```
Epoch 40/50 | Train MSE: 0.01355 | Val MSE: 0.01348 | Test MSE: 0.01304
Epoch 50/50 | Train MSE: 0.00860 | Val MSE: 0.00856 | Test MSE: 0.00823
→ Digit-3 Test MSE (60 hidden units): 0.008889
```

```
=====
```

Training autoencoder with 120 hidden units

Optimizer: Adam | LR: 0.001 | Epochs: 50 | Batch size: 256

```
=====
```

```
Epoch 1/50 | Train MSE: 0.07457 | Val MSE: 0.05887 | Test MSE: 0.05895
Epoch 10/50 | Train MSE: 0.01535 | Val MSE: 0.01465 | Test MSE: 0.01431
Epoch 20/50 | Train MSE: 0.00787 | Val MSE: 0.00782 | Test MSE: 0.00755
Epoch 30/50 | Train MSE: 0.00542 | Val MSE: 0.00549 | Test MSE: 0.00529
Epoch 40/50 | Train MSE: 0.00416 | Val MSE: 0.00426 | Test MSE: 0.00410
Epoch 50/50 | Train MSE: 0.00334 | Val MSE: 0.00345 | Test MSE: 0.00332
→ Digit-3 Test MSE (120 hidden units): 0.003594
```

8. Learning Curves — MSE vs. Epoch (Q1 — Part 1)

Below we plot the **learning curves** for each bottleneck size. Each subplot shows three curves:

- **Blue (solid):** Training MSE
- **Orange (dashed):** Validation MSE
- **Green (dotted):** Test MSE (treated like a second validation set, as instructed)

What to look for:

- All three curves should decrease together, indicating good generalization
- If training MSE keeps dropping but val/test plateau or increase, the model is overfitting
- Smaller bottlenecks (e.g., 5 units) converge to a higher MSE floor because they can't represent all image details
- Larger bottlenecks (e.g., 120 units) achieve lower MSE because they have more capacity to preserve information

```
In [9]: epochs = range(1, NUM_EPOCHS + 1)
n_plots = len(HIDDEN_UNITS_LIST)
n_cols = 3
n_rows = (n_plots + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(6 * n_cols, 5 * n_rows))
axes = np.array(axes).flatten()

for idx, h in enumerate(HIDDEN_UNITS_LIST):
    _, train_l, val_l, test_l = all_results[h]
    ax = axes[idx]

    ax.plot(epochs, train_l, color='steelblue', linewidth=1.8, label='Train')
    ax.plot(epochs, val_l, color='darkorange', linewidth=1.8,
            linestyle='--', label='Validation')
    ax.plot(epochs, test_l, color='seagreen', linewidth=1.8,
            linestyle=':', label='Test')
```

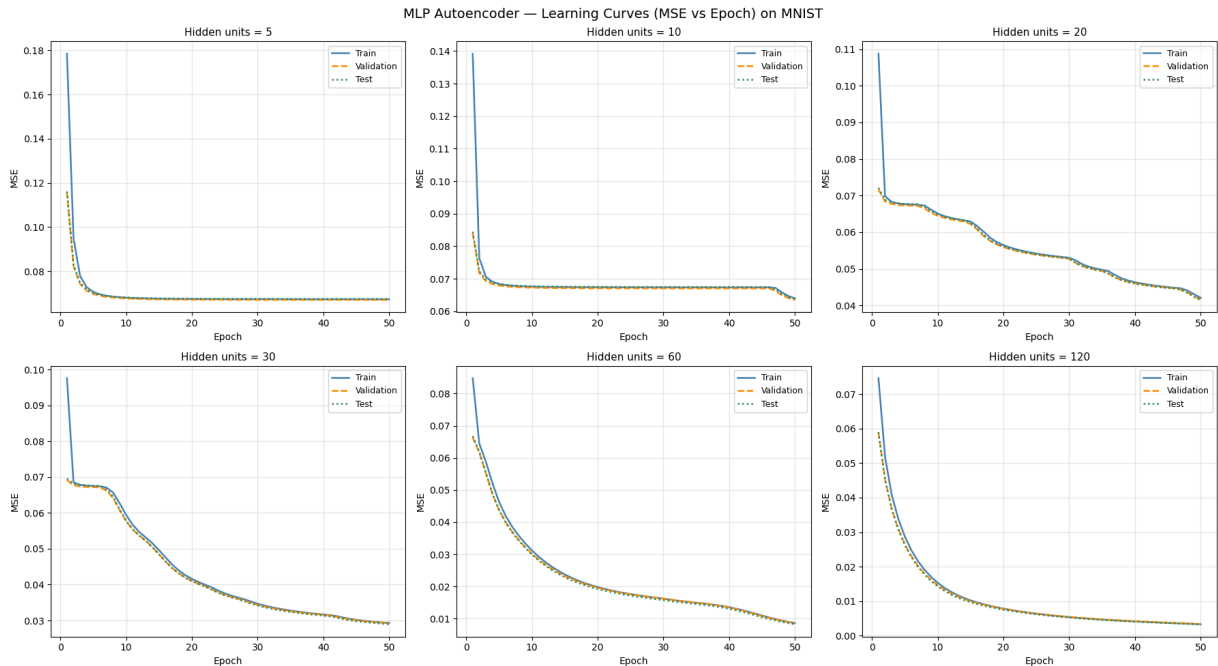
```

ax.set_title(f'Hidden units = {h}', fontsize=11)
ax.set_xlabel('Epoch')
ax.set_ylabel('MSE')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

for idx in range(n_plots, len(axes)):
    axes[idx].set_visible(False)

fig.suptitle('MLP Autoencoder — Learning Curves (MSE vs Epoch) on MNIST', fontsize=
plt.tight_layout()
plt.show()

```



Observations — Learning Curves

- **All models converge smoothly** — training, validation, and test MSE decrease together without significant divergence, indicating no overfitting. This is expected for a simple autoencoder on MNIST.
- **Smaller bottlenecks (5, 10 units)** converge to higher MSE values because they are forced to compress 784 dimensions into very few features, losing fine details.
- **Larger bottlenecks (60, 120 units)** achieve much lower MSE, as they retain more information from the input.
- **Validation and test curves closely track each other**, confirming that the test set behaves similarly to the validation set (good data split).
- **Most learning happens in the first 10–20 epochs**, with diminishing improvements afterward.

9. Digit "3" Reconstruction MSE vs. Bottleneck Size (Q1 — Part 1)

This bar chart summarizes how well each model reconstructs the digit "3" specifically. The assignment asks us to evaluate MSE for digit 3 with different hidden unit counts.

We expect MSE to decrease as the number of hidden units increases — more capacity means the autoencoder can encode finer details of the digit's shape.

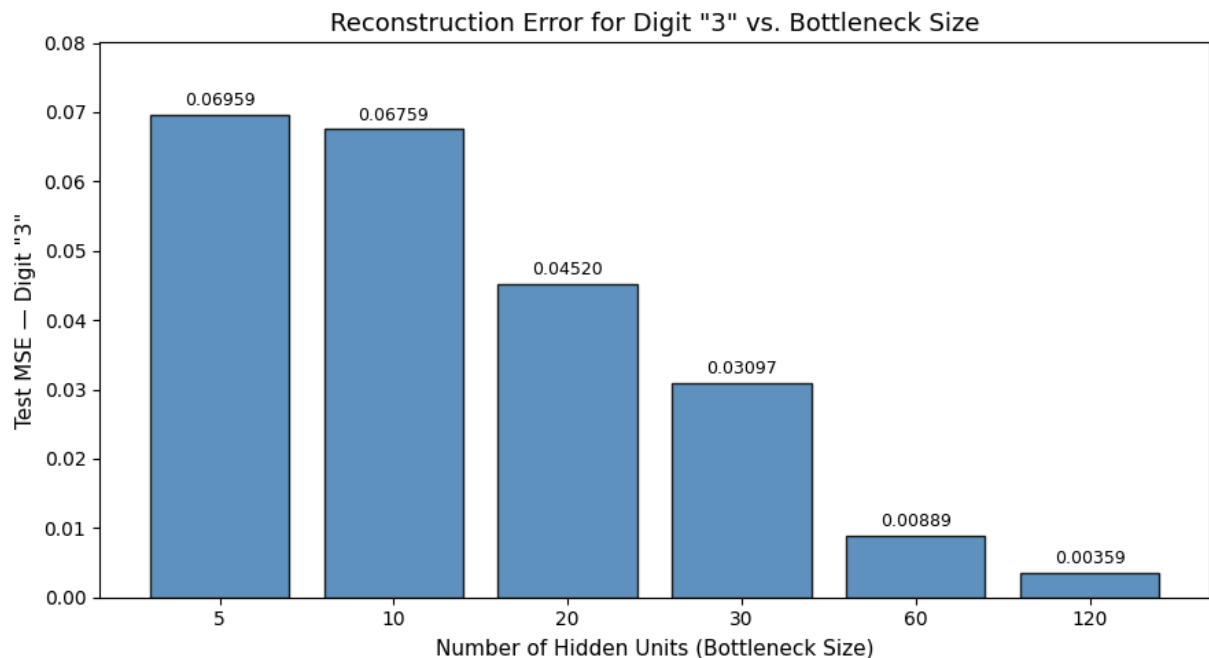
```
In [10]: hs = list(digit3_mse.keys())
mSES = list(digit3_mse.values())

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.bar([str(h) for h in hs], mSES, color='steelblue', edgecolor='black', alp

for bar, mse_val in zip(bars, mSES):
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + max(mSES) * 0.01,
            f'{mse_val:.5f}', ha='center', va='bottom', fontsize=9)

ax.set_xlabel('Number of Hidden Units (Bottleneck Size)', fontsize=11)
ax.set_ylabel('Test MSE — Digit "3"', fontsize=11)
ax.set_title('Reconstruction Error for Digit "3" vs. Bottleneck Size', fontsize=13)
ax.set_ylim(0, max(mSES) * 1.15)
plt.tight_layout()
plt.show()

# Print numeric results
print("\nDigit '3' Reconstruction MSE by bottleneck size:")
for h, m in digit3_mse.items():
    print(f" {h:3d} hidden units → MSE = {m:.6f}")
```



Digit '3' Reconstruction MSE by bottleneck size:

```
5 hidden units → MSE = 0.069595
10 hidden units → MSE = 0.067588
20 hidden units → MSE = 0.045197
30 hidden units → MSE = 0.030969
60 hidden units → MSE = 0.008889
120 hidden units → MSE = 0.003594
```

Observations — Digit 3 MSE

- **Clear inverse relationship** between bottleneck size and reconstruction error — more hidden units produce lower MSE.
- The **biggest improvements** come from increasing from 5 to 30 units. Beyond 30 units, returns diminish.
- With 120 hidden units, the model can reconstruct digit 3 with very low error, preserving most of the fine stroke details.

10. Reconstruction Gallery — Digit "3" (Q1 — Part 1)

Here we visually compare the original digit "3" against its reconstruction from each model. This gives an intuitive sense of how compression quality improves with bottleneck size.

With only 5 hidden units, the reconstruction is blurry and loses detail. With 120 hidden units, the reconstruction is nearly indistinguishable from the original.

```
In [11]: # Find the first digit-3 in the test set
digit3_idx = np.where(test_labels == 3)[0][0]
sample_img = torch.from_numpy(test_images[digit3_idx]).unsqueeze(0).to(device)

n_models = len(HIDDEN_UNITS_LIST)
fig, axes = plt.subplots(1, n_models + 1, figsize=(3.0 * (n_models + 1), 3.2))

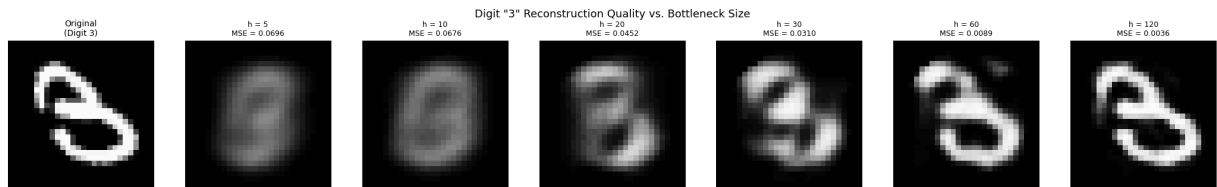
# Original image
axes[0].imshow(test_images[digit3_idx].reshape(28, 28), cmap='gray', vmin=0, vmax=1)
axes[0].set_title('Original\n(Digit 3)', fontsize=10)
axes[0].axis('off')

# Reconstructions from each model
for idx, h in enumerate(HIDDEN_UNITS_LIST):
    model = all_results[h][0]
    model.eval()
    with torch.no_grad():
        recon_np = model(sample_img).cpu().numpy().reshape(28, 28)

    axes[idx + 1].imshow(recon_np, cmap='gray', vmin=0, vmax=1)
    axes[idx + 1].set_title(f'h = {h}\nMSE = {digit3_mse[h]:.4f}', fontsize=9)
    axes[idx + 1].axis('off')

fig.suptitle('Digit "3" Reconstruction Quality vs. Bottleneck Size', fontsize=13)
```

```
plt.tight_layout()
plt.show()
```



11. Encoder Weight Visualization (Q1 — Part 2)

The encoder's weight matrix W_1 has shape $(N, 784)$, where N is the number of hidden units. **Row i** of this matrix contains the 784 weights that connect every input pixel to hidden neuron i .

By reshaping row i from a 784-dimensional vector to a **28×28 image**, we can visualize what each hidden neuron has learned to detect:

- **Bright pixels** indicate strong positive weights — the neuron responds when those pixels are active
- **Dark pixels** indicate strong negative weights or near-zero weights

With **10 hidden units**, the network must compress all of MNIST into just 10 features, so each weight image tends to look like a broad, prototypical digit shape. With **20 hidden units**, the features become more fine-grained and capture more specific patterns.

This visualization corresponds to the course notes on autoencoders (pp. 78–79).

Encoder Weights — 10 Hidden Units

```
In [12]: h = 10
model = all_results[h][0]
W = model.get_encoder_weights() # shape: (10, 784)

# Normalize each weight image individually to [0, 1] for display
W_min = W.min(axis=1, keepdims=True)
W_max = W.max(axis=1, keepdims=True)
W_norm = (W - W_min) / (W_max - W_min + 1e-8)

n_cols_w = min(h, 10)
n_rows_w = (h + n_cols_w - 1) // n_cols_w

fig, axes = plt.subplots(n_rows_w, n_cols_w, figsize=(n_cols_w * 1.8, n_rows_w * 1.8))
axes = np.array(axes).flatten()

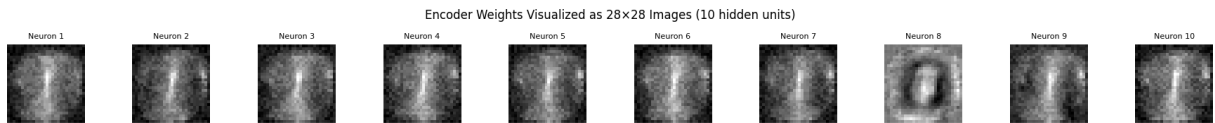
for i in range(h):
    weight_img = W_norm[i].reshape(28, 28)
    axes[i].imshow(weight_img, cmap='gray', vmin=0, vmax=1)
    axes[i].set_title(f'Neuron {i + 1}', fontsize=8)
    axes[i].axis('off')
```

```

for i in range(h, len(axes)):
    axes[i].set_visible(False)

fig.suptitle(f'Encoder Weights Visualized as 28x28 Images ({h} hidden units)', font
plt.tight_layout()
plt.show()

```



Encoder Weights — 20 Hidden Units

```

In [13]: h = 20
model = all_results[h][0]
W = model.get_encoder_weights() # shape: (20, 784)

W_min = W.min(axis=1, keepdims=True)
W_max = W.max(axis=1, keepdims=True)
W_norm = (W - W_min) / (W_max - W_min + 1e-8)

n_cols_w = min(h, 10)
n_rows_w = (h + n_cols_w - 1) // n_cols_w

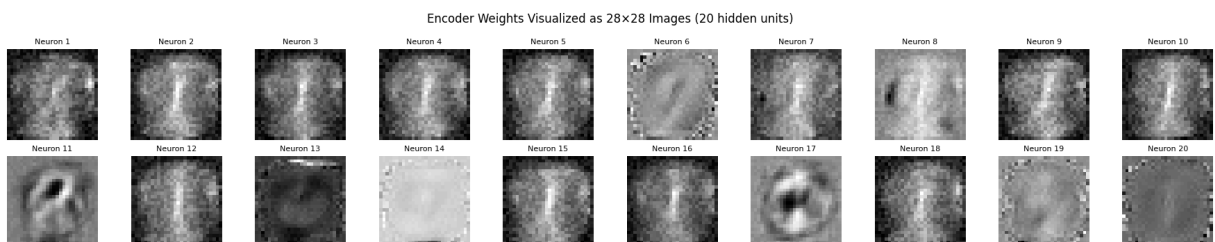
fig, axes = plt.subplots(n_rows_w, n_cols_w, figsize=(n_cols_w * 1.8, n_rows_w * 1.
axes = np.array(axes).flatten()

for i in range(h):
    weight_img = W_norm[i].reshape(28, 28)
    axes[i].imshow(weight_img, cmap='gray', vmin=0, vmax=1)
    axes[i].set_title(f'Neuron {i + 1}', fontsize=8)
    axes[i].axis('off')

for i in range(h, len(axes)):
    axes[i].set_visible(False)

fig.suptitle(f'Encoder Weights Visualized as 28x28 Images ({h} hidden units)', font
plt.tight_layout()
plt.show()

```



Observations — Encoder Weights

- **10 hidden units:** Each neuron's weight pattern resembles a broad, blurry digit or digit fragment. Since the network has only 10 features to represent all 10 digit classes, each

neuron must capture general structural patterns shared across multiple digits (e.g., vertical strokes, loops, horizontal bars).

- **20 hidden units:** The weight patterns become more fine-grained and localized. With double the capacity, the network can dedicate neurons to more specific features — individual stroke orientations, specific curve patterns, or particular digit sub-regions. This additional specificity is what enables lower reconstruction error.

These visualizations confirm that the autoencoder is learning meaningful, interpretable features rather than random noise — exactly what we would expect from a well-trained dimensionality reduction model.

Summary

In this assignment, we built and trained 3-layer MLP autoencoders on the MNIST dataset to study dimensionality reduction. Key findings:

1. **Reconstruction quality scales with bottleneck size** — increasing hidden units from 5 to 120 monotonically decreases MSE, with the most significant gains between 5 and 30 units.
2. **No overfitting observed** — training, validation, and test learning curves track closely for all configurations, indicating good generalization. This is expected for a simple linear-nonlinear autoencoder on a relatively simple dataset.
3. **Encoder weights reveal learned features** — with 10 units, the network learns broad digit-like templates; with 20 units, the features become more localized and specific, enabling finer reconstruction.
4. **Hyperparameters used:** Adam optimizer (LR=0.001), MSE loss, Sigmoid activations, batch size 256, 50 training epochs, 80/20 train/validation split.